

Markdown & Quarto for clinical pharmacology

Plain-text documents, slide decks, and equations

Markus Hovd

Oslo University Hospital

26 September 2026

Agenda

Every example shows **the source you type** and, directly below it, **what it produces**.

1. **Why plain text** — with examples
2. **Markdown essentials**
3. **Documents with Quarto**
4. **Presentations**
5. **Equations**
6. **Your turn** — reproduce a document

Why plain text?

- ▶ **Portable** — a .qmd is text; it survives software changes
- ▶ **Version-controlled** — every edit is a dated, attributed line in the history
- ▶ **Reproducible** — numbers, tables and figures are computed from your input files
- ▶ **One source, many outputs** — HTML, PDF, Word, slides
- ▶ **Scriptable** — 300 input files become 300 reports

Version control: what changed, and when

```
git diff report.qmd
```

```
--- a/report.qmd
+++ b/report.qmd
@@ -12,7 +12,7 @@
-The mean trough was 5.4 mg/L.
+The mean trough was 6.1 mg/L.

-Sampling was pre-dose in all patients.
+Sampling was pre-dose in all but two patients.
```

Two edited lines, both visible, both attributable. A .docx is a binary blob: `git diff` on it says only *Binary files differ*, which is why “track changes” is the best Word can do.

Reproducible: numbers come from the input files

```
The mean trough in this cohort is `r mean(conc$c_trough)` mg/L.
```

The mean trough in this cohort is 5.8 mg/L.

Change a value in the input file, re-render, and every number, table and figure that depends on it updates together. Nobody copies a figure by hand, so nobody copies it wrongly.

Anatomy of a .qmd file

```
---
title: "Tacrolimus troughs, Q3"
author: "Your Name"
format: html          # html | pdf | docx | revealjs
---

Prose goes here in Markdown, including  $C_{ss} = R_0 / CL$ .

```{r}
mean(c(4.1, 5.6, 6.3)) # runs at render
```
```

YAML header — metadata and output settings · **Body** — Markdown · **Code chunks** — run at render time. Positron ships Quarto: use **Render/Preview**, or quarto preview slides.qmd.

Part 1 — Markdown essentials

Headings create structure

```
# Level 1 - a part of the document  
## Level 2 - a section  
### Level 3 - a subsection
```

Level 1 — a part of the document

Level 2 — a section

Level 3 — a subsection

Text: emphasis, paragraphs, breaks

```
*italic*  **bold**  ***both***  ~~struck~~  `inline code`
```

A blank line makes a new paragraph. A single line break inside one is ignored.

End a line with two spaces,
and you get a hard break.

italic **bold** *both* struck inline code

A blank line makes a new paragraph. A single line break inside one is ignored.

End a line with two spaces,
and you get a hard break.

Bulleted lists

- Tacrolimus
- Ciclosporin
 - whole blood
 - trough concentration
- Everolimus

- ▶ Tacrolimus
- ▶ Ciclosporin
 - ▶ whole blood
 - ▶ trough concentration
- ▶ Everolimus

Numbered lists

```
1. Draw the trough  
1. Centrifuge  
1. Freeze the sample
```

1. Draw the trough
2. Centrifuge
3. Freeze the sample

Quarto numbers the list for you. The source above writes 1. on every line and still renders 1, 2, 3, so inserting an item in the middle never means renumbering by hand.

Task lists

- [x] Ethics approval
- [x] Assay validation
- [] Recruit 40 patients

- ☒ Ethics approval
- ☒ Assay validation
- ☐ Recruit 40 patients

Links and images

[Quarto documentation] (<https://quarto.org>)

![Concentration-time curve](images/pk-curve.png){#fig-curve}

See @fig-curve.

Quarto documentation

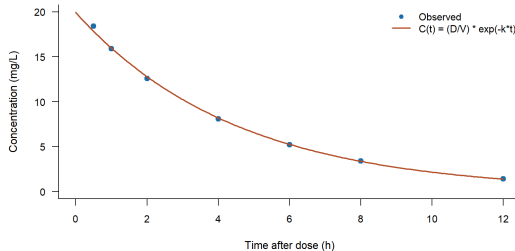


Figure 1: Concentration–time curve

Tables

| Time (h) | Conc. (mg/L) |
|----------|--------------|
| 1 | 12.4 |
| 4 | 8.1 |

: Vancomycin concentrations {#tbl-conc}

Table 1: Vancomycin concentrations

| Time (h) | Conc. (mg/L) |
|----------|--------------|
| 1 | 12.4 |
| 4 | 8.1 |

Quotes and callouts

```
> Quote a guideline, verbatim.
```

```
::: {.callout-tip}
## Practical tip
A callout groups advice.
:::
```

Quote a guideline, verbatim.

 Practical tip

A callout groups advice.

Other callout types: `.callout-note`, `.callout-important`, `.callout-caution`, `.callout-warning`.

Code blocks

Inline code like ``CL = D/AUC`` sits in a sentence.

```
```python
auc = sum((c[i] + c[i+1]) / 2 * (t[i+1] - t[i])
 for i in range(len(c) - 1))
```
```

Inline code like `CL = D/AUC` sits in a sentence.

```
auc = sum((c[i] + c[i+1]) / 2 * (t[i+1] - t[i])
          for i in range(len(c) - 1))
```

Fenced blocks get syntax highlighting and a copy button. To *show* a fenced block, wrap it in one more backtick — four around three, as this slide does.

Markdown pitfalls worth knowing on day one

- ▶ `*` and `_` are **always** formatting characters: `AUC_MIC` becomes `AUC*MIC`. Write `AUC\_MIC` or `AUC-MIC`.
- ▶ One blank line between blocks — two spaces and no blank line will surprise you.
- ▶ Do not indent ordinary paragraphs: 4 spaces means “code”.
- ▶ Front matter only counts at the very top of the file.

Part 2 — Documents with Quarto

The YAML header — where the document is configured

```
---  
title: "Vancomycin AUC monitoring"  
subtitle: "Pharmacy department"  
author: "Your Name"  
date: today  
format:  
  html:  
    toc: true  
    number-sections: true  
---
```

Vancomycin AUC monitoring Pharmacy department Your Name · 24 September 2026

Parameters: one document, many versions

```
---  
title: "Trough report"  
params:  
  data_file: "data/cohort-a.csv"  
  centre: "Oslo University Hospital"  
  recipient: "Dr. Smith"  
---
```

Use them in the text and in your code as `params$name`:

```
Dear `r params$recipient` - this report is from `r params$centre`.  
  
```{r}  
conc <- read.csv(params$data_file)
```
```

Overriding parameters at render time

```
quarto render report.qmd -P centre:"Trondheim"  
quarto render report.qmd -P data_file:"data/cohort-b.csv"
```

One source, many versions — a **new input file**, a different **centre**, a **letter** to a different recipient. No editing, no copies of the document.

Computed numbers: code chunks

```
The mean trough in this cohort is `r mean(conc$c_trough)` mg/L.
```

```
```${r}  
#| label: fig-conc
#| fig-cap: "Concentration-time profile."
plot(conc$time, conc$value, pch = 19)
```
```

- ▶ ``r ... `` — **inline** code: the value is written into the sentence at render time
- ▶ `#| label: fig-conc` — names the figure, so you can write `see @fig-conc`

Never type a number twice: let the document compute it.

Cross-references

Annotate the thing itself, then refer to it anywhere else — the numbers are generated.

| Annotate | Reference | Result |
|--|---------------------------|-------------|
| <code># Methods {#sec-methods}</code> | <code>@sec-methods</code> | Section 2.3 |
| <code>{#fig-curve}</code> | <code>@fig-curve</code> | Figure 1 |
| <code>: Caption {#tbl-conc}</code> | <code>@tbl-conc</code> | Table 1 |
| <code>\$\$ y = x^2 \$\$ {#eq-fit}</code> | <code>@eq-fit</code> | Equation 1 |

Citations in text

```
The `pharmsol` library supports pharmacokinetic and pharmacodynamic modeling and simulation [@Otálvaro2026].
```

The pharmsol library supports pharmacokinetic and pharmacodynamic modeling and simulation (Otálvaro et al. 2026).

The citation key in square brackets becomes a formatted in-text citation.

The .bib file

```
@article{Otálvaro2026,  
  doi      = {10.21105/joss.10580},  
  year     = {2026},  
  author    = {Otálvaro, Julián D. and Hovd, Markus  
              and Yamada, Walter M. and Neely, Michael},  
  title     = {pharmsol: A high-performance Rust library for  
              pharmacokinetic and pharmacodynamic modeling and simulation},  
  journal   = {Journal of Open Source Software}  
}
```

bibliography: references.bib in the YAML; csl: vancouver.csl picks the style.

One source, many outputs

```
quarto render report.qmd           # the format in its YAML
quarto render report.qmd --to docx # for co-authors who live in Word
quarto render report.qmd --to pdf  # needs LaTeX, or --to typst
```

| Output | Use it for | Needs |
|--------|-----------------------------------|----------------|
| HTML | sharing, reading, reproducibility | nothing |
| PDF | submission, print, posters | LaTeX or Typst |
| DOCX | co-authors, journals, reviewers | nothing |
| beamer | presentations from .qmd | LaTeX |

This deck is the demonstration: `slides.qmd` is one file, and the PDF you are reading is what it renders to.

Part 3 — Presentations

How Markdown becomes slides

| In the file | On the screen |
|-----------------------|---------------------------------------|
| The YAML header | The title slide |
| # Part 1 - Essentials | A section divider |
| ## Slide title | A new slide |
| ### Sub-title | A sub-heading <i>inside</i> the slide |

Headings paginate the deck: a **##** you meant as an in-text heading starts a brand new slide. `slide-level: 3` in the YAML makes **###** start slides too.

Layout toolkit

| Tool | Syntax |
|---------------------|--|
| Source above result | a fenced block, then the result |
| Two columns | ::: {.columns} + ::: {.column width="50%"} |
| Section divider | # Part 1 - Title |
| Deeper nesting | slide-level: 3 makes ### a slide |

`{.smaller}`, `{.incremental}`, `{.center}` and background colours are **reveal.js only** — in a PDF they are ignored, silently, with no error. So fit a slide by cutting words rather than shrinking the type.

Presenting and exporting

- ▶ `quarto render slides.qmd` — builds `slides.pdf`, 16:9, with a footer and page numbers
- ▶ `quarto preview slides.qmd` — your rehearsal loop, re-rendering on every save (in Positron, the **Preview** button in the action bar does the same)
- ▶ `::: {.notes}` blocks are **not** rendered to the PDF. Keep them in the source as your script, or delete them — either way the audience never sees them
- ▶ A PDF needs a LaTeX toolchain once: `quarto install tinytex`

Part 4 — Equations

Inline or display

The trough was $C_{\min} = 4.2$ mg/L.

\$\$

$$C(t) = \frac{D}{V_d} e^{-k_e t}$$

\$\$

The trough was $C_{\min} = 4.2$ mg/L.

$$C(t) = \frac{D}{V_d} e^{-k_e t}$$

$\dots$ is **inline** — part of the sentence. $\dots$ is **display** — its own centred line.

Symbols: subscripts, fractions, operators

| Type this | You get | Type this | You get |
|-------------------------------|--------------------|-------------------------------|--------------------|
| <code>x_{ss}, x^{2}</code> | x_{ss}, x^2 | <code>\frac{a}{b}</code> | $\frac{a}{b}$ |
| <code>\ln 2, \log_{10}</code> | $\ln 2, \log_{10}$ | <code>\sqrt{x}</code> | $\sqrt{x}$ |
| <code>\sum_{i=1}^n</code> | $\sum_{i=1}^n$ | <code>\int_0^\infty</code> | $\int_0^\infty$ |
| <code>\lambda_z, \mu</code> | λ_z, μ | <code>\Delta, \partial</code> | Δ, ∂ |
| <code>\geq \leq \neq</code> | $\geq \leq \neq$ | <code>\approx \propto</code> | $\approx \propto$ |
| <code>\left(\right)</code> | $()$ | <code>\mathrm{mg/L}</code> | mg/L |

Multi-line equations with aligned

```
$$\begin{aligned}C(t) &= C_0 e^{-k_e t} \\\text{AUC} &= \frac{C_0}{k_e} \\\end{aligned}$$
```

$$C(t) = C_0 e^{-k_e t}$$
$$AUC = \frac{C_0}{k_e}$$

`&` marks the alignment point and `\\` ends the line.

Piecewise equations with cases

```
$$C(t) =  
\begin{cases}  
\frac{R_0}{CL}\left(1 - e^{-k_e t}\right) & t \leq T \\\br/>C_{ss}e^{-k_e (t-T)} & t > T  
\end{cases}$$
```

$$C(t) = \begin{cases} \frac{R_0}{CL} (1 - e^{-k_e t}) & t \leq T \\ C_{ss} e^{-k_e (t-T)} & t > T \end{cases}$$

cases gives a piecewise definition — one line per case, condition after &.

Units, spacing and text

```
$C = 8.1\ \mathrm{mg/L}$
```

```
$R_0 = 1\ \mathrm{g/h}$
```

```
$\text{steady state}$
```

```
$95\%$ CI: $7.2$-$9.0$
```

$C = 8.1 \text{ mg/L}$

$R_0 = 1 \text{ g/h}$

steady state

95% CI: 7.2–9.0

Numbered equations and cross-references

```
$$  
AUC_{0 \to t_n} = \sum_{i=1}^{n-1}   
  \frac{C_i + C_{i+1}}{2} \Delta t_i  
$$ {#eq-auc}
```

The exposure in @eq-auc is estimated numerically.

$$AUC_{0 \rightarrow t_n} = \sum_{i=1}^{n-1} \frac{C_i + C_{i+1}}{2} \Delta t_i$$

{#eq-auc} gives the equation a number; @eq-auc links to it as “equation 1”.
Renumbering is automatic — add an equation above and every reference follows.

Equation pitfalls

| Symptom | Cause & fix |
|--|---|
| <code>C_max</code> renders as $C_m ax$ | Subscripts need braces: <code>C_{max}</code> |
| <code>ln(2)</code> renders as $ln(2)$ | Use the command: <code>\ln(2)</code> |
| Units come out italic | Wrap them: <code>\mathrm{mg/L}</code> |
| The whole paragraph turns into math | Unmatched <code>\$</code> — count your dollars |
| <code>%</code> swallows the rest of the equation | Escape it: <code>\%</code> |
| Equation runs off the side of the slide | MathML cannot wrap — split it with <code>aligned</code> |

Part 5 — Your turn

Your turn — reproduce a document

Brief: exercises/task.pdf · **Target:** reproduce-target.pdf · **Input:** data/patient-concentrations.csv

Build a .qmd that renders like the target.

- ☐ A YAML header — title, author, date: today, PDF output
- ☐ Numbered sections using ## headings
- ☐ A paragraph with **bold**, *italic* and an inline equation
- ☐ A **numbered display equation**, labelled and referred to as @eq-...
- ☐ An **R code chunk** that reads the CSV and draws a figure
- ☐ That figure **cross-referenced** as @fig-...
- ☐ A table with a caption and a label, referred to as @tbl-...

Stuck? reproduce-solution.pdf is the same document with every code chunk shown.

Where to go next

- ▶ **This repo:** `cheatsheet.qmd` — every syntax you saw, on one page
- ▶ **Positron:** <https://positron.posit.co> — the IDE, with Quarto built in
- ▶ **Quarto:** <https://quarto.org/docs/guide/>
- ▶ **Markdown:** <https://commonmark.org/help/>
- ▶ **Equations:** <https://en.wikibooks.org/wiki/LaTeX/Mathematics>
- ▶ **Symbols:** <https://detexify.kirelabs.org> — draw a symbol, get the command

References

Otálvaro, Julián D., Markus Hovd, Walter M. Yamada, and Michael Neely. 2026. “Pharmsol: A High-Performance Rust Library for Pharmacokinetic and Pharmacodynamic Modeling and Simulation.” *Journal of Open Source Software*, ahead of print. <https://doi.org/10.21105/joss.10580>.

Thank you

Questions, or a `.qmd` you want me to look at?

Markus Hovd — Oslo University Hospital

`mhovd@ous-hf.no`

Slides: `slides.qmd` in this repository — render them yourself and edit them.